DocketPilot: An Agentic AI System for Indian Legal Research, Drafting, Anti-Hallucination Verification and Case Management

Srigaran N
[Department], [Institution], [City], India, srigaran2020@gmail.com
[Co-author name]
[Department], [Institution], [City], India, [email]
Dr. Saravanan R
[Department], [Institution], [City], India, [email]

***Abstract*—Large language models can draft legal text fluently, yet they routinely cite judgments that do not exist or that say something else, and sending client papers to a hosted model exposes personal data. We present DocketPilot, an agentic system for Indian legal practice that couples retrieval over Supreme Court and High Court judgments and Central Acts with a citation verifier that checks, corrects or removes every authority before an answer is released. Client identifiers are masked on the user's machine before any model call, and hearing dates read from case papers populate a built-in calendar. Seven agents coordinated by a stateful graph handle document analysis, research, drafting, verification, risk assessment and deadline monitoring, with a human approval step before any document is finalised. On a benchmark of 756 corrupted and 156 genuine citations the verifier reaches 99.6% recall and 99.5% F1. In an ablation with a single open-weight model, the share of unreliable citations falls from 16.1% when the model answers from memory to 0.0% with the full pipeline, while a research answer costs 87.2% less than giving the same model the full judgments. LIME attributions show that evidence selection is driven by the legally decisive terms of each query.**

***Index Terms*—legal AI, retrieval-augmented generation, hallucination, citation verification, multi-agent systems, privacy, explainable AI.**

I. Introduction

Indian courts carry a pending caseload in the tens of millions, and much of an advocate's time goes into locating precedent, checking that a section is in force on the relevant date, and preparing routine filings. General-purpose language models promise to shorten this work, but two properties make them hazardous in a courtroom. First, they hallucinate authority: independent studies report fabricated or misattributed citations in a substantial share of legal answers, even from commercial research tools [2], [3]. Second, a hosted model receives whatever text it is given, which for a practising advocate means names, addresses and identity numbers of clients.

Recent agentic legal systems focus on throughput and cost. LegalMind [1], for example, reports cost and processing-time gains from a fine-tuned reasoning model, but does not report citation reliability. Work on Indian legal text has concentrated on prediction and understanding benchmarks [23]–[25] rather than on systems that a lawyer can trust to cite correctly. A gap therefore remains between fluent generation and verifiable legal output for Indian law.

This paper describes DocketPilot, a working system that closes this gap with three design choices: (i) every legal statement produced by the model must be tied to a retrieved passage or statute section and is re-checked after generation; (ii) failures are repaired automatically rather than merely flagged, so that no unverified citation reaches the user; and (iii) all identifying data is masked locally before inference. The contributions are:

- an end-to-end agentic architecture for Indian legal research, drafting and case management built around a released-only-if-verified rule;
- a nine-step citation verifier with automatic correction, evaluated on a corrupted-citation benchmark and in a layer-by-layer ablation;
- a measured comparison of cost, speed and hallucination against the same model without the pipeline and against published systems; and
- an explainability analysis of evidence selection using LIME [12].

II. Related Work

A. Hallucination in legal AI

Dahl et al. [3] queried general models on verifiable facts about US federal cases and observed hallucination rates of 69% for GPT-3.5 and 88% for Llama 2. Magesh et al. [2] evaluated retrieval-backed commercial products and still found 17% (Lexis+ AI) and 33% (Westlaw AI-Assisted Research) of answers to be incorrect or misgrounded. LegalCiteBench [5] shows that closed-book citation recovery remains below 7/100 for the strongest of 21 models. Ovcharov [6] cautions that measured rates depend on the coverage of the reference database, a point we account for in Section V.

B. Citation checking and retrieval

Liu et al. [4] benchmark citation-error detection on excerpts of US briefs; their best agentic configuration reaches 84.4% recall and the best F1 is 68.8%. Retrieval-augmented generation [11] grounds answers in documents, and hybrid lexical–dense retrieval with reciprocal rank fusion [13] and cross-encoder re-ranking [15] is now a common recipe. For

legal documents, Hany et al. [8] show that navigating a structured index instead of injecting the whole corpus can cut cost by 25–67.5%.

C. Indian legal NLP

ILDC [24] and IL-TUR [23] provide judgment-prediction and understanding tasks, and InLegalBERT [25] adapts encoders to Indian legal text. Falkor-IRAC [7] constrains generation with a graph of Indian judgments but reports qualitative results on a small corpus. NyayaAI [26] routes queries through a multi-agent pipeline with FAISS retrieval and a compliance check, reporting 74% retrieval precision and 72% response accuracy; AQgR [27] adds question-guided retrieval and structured summaries for prior-case search, reaching a MAP of 0.36 on FIRE 2019; and a domain-partitioned hybrid RAG with a knowledge graph [28] passes 70% of 40 synthetic questions judged by an LLM. None of these verify the citations they emit. To our knowledge no prior Indian system combines retrieval, generation, automatic citation repair, local privacy masking and case management in one deployable application.

TABLE I. POSITIONING AGAINST RELATED SYSTEMS (CITE: CITATION CHECK; FIX: AUTOMATIC REPAIR; PII: PRIVACY MASKING; CAL.: HEARING CALENDAR; ✓ REPORTED IN THE CITED SOURCE; – NOT REPORTED)

System	Scope	RAG	Cite	Fix	PII	Cal.
DocketPilot	India	✓	✓	✓	✓	✓
LegalMind [1]	General	–	–	–	–	–
Lexis+ AI [2]	US	✓	–	–	–	–
Westlaw AI-AR [2]	US	✓	–	–	–	–
Agentic checker [4]	US	✓	✓	–	–	–
Falkor-IRAC [7]	India	✓	✓	–	–	–
NyayaAI [26]	India	✓	–	–	–	–
NAVINDEX [8]	Contracts	✓	–	–	–	–

III. SYSTEM ARCHITECTURE

Fig. 1 shows the end-to-end flow. A single-page web client talks to a FastAPI service; PostgreSQL stores the knowledge base, user data and agent checkpoints, and Qdrant stores dense vectors. The only generative model is the open-weight gpt-oss-120b [18], served through Groq; it never receives unmasked text.

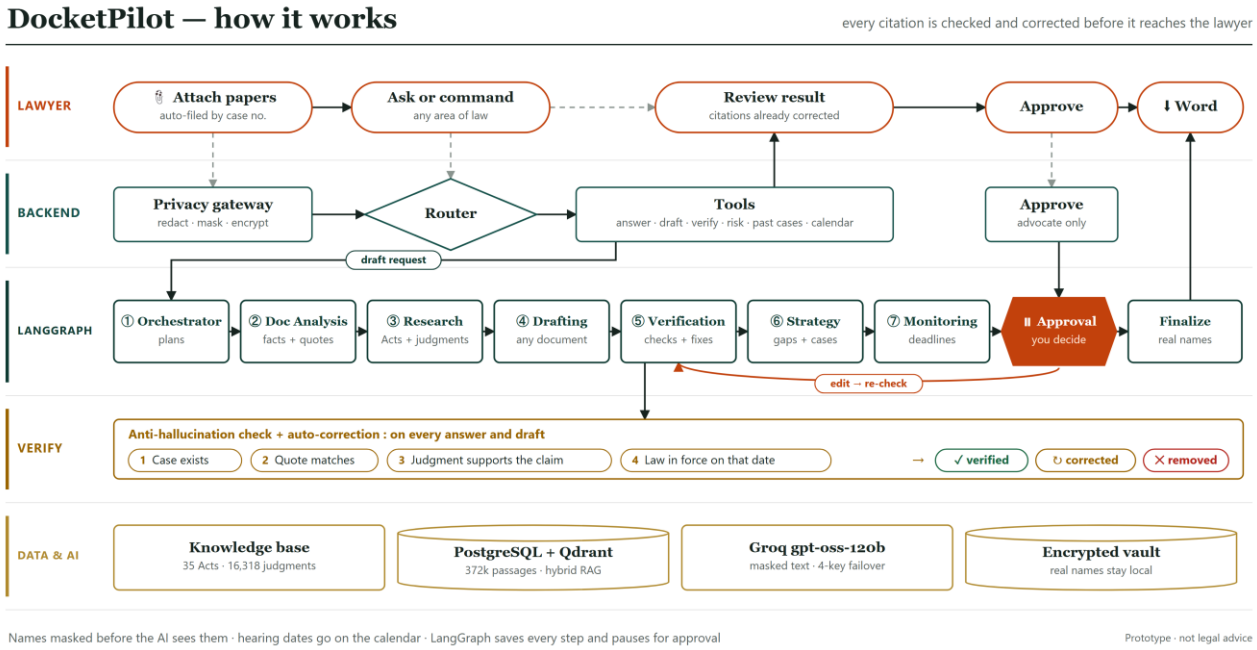


Fig. 1. DocketPilot workflow across five lanes: the lawyer, the backend, the agent graph, verification, and data and models.

A. Knowledge base

Judgments come from the open Indian Supreme Court and High Court collections released under CC-BY-4.0, and statutes from India Code. The store holds 48,168 Supreme Court cases for citation resolution (1950–2026), 16,318 judgments with full text, 79 Central Acts with 8,558 sections and their amendment footnotes, and 430,882 passages, of which 372,122 are embedded with bge-small-en-v1.5 [14]. Headnotes are stored as a separate segment and never used as evidence.

B. Privacy gateway

Before any model call, a local recogniser replaces identifiers with consistent placeholders (PERSON_1, ORG_2): rule patterns with checksums for Aadhaar, PAN, bank, IFSC, telephone and vehicle numbers, and a spaCy [22] named-entity model for names and organisations. The lawyer reviews the masked text before it is stored. The placeholder map is kept per case in a Fernet-encrypted vault, and real names are restored only on screen and in the approved document.

C. Agents

Seven agents run as nodes of a LangGraph [19] state graph that checkpoints after every step: an orchestrator selects the workflow; document analysis extracts facts and keeps a fact only when its supporting quote is found in the papers; research gathers statute sections and re-ranked passages; drafting writes the document with a source tag on every legal statement; verification checks and repairs it; strategy and risk lists missing facts and similar past cases; and monitoring derives statutory deadlines. The graph then pauses for the lawyer's approval. A rule-based router exposes the same tools through one conversational interface, and dates extracted from uploaded papers, each anchored to a quote, are filed to a per-case calendar.

D. Retrieval

Retrieval proceeds from exact to fuzzy: exact citation lookup, statute lookup as on a date, statute search by topic, PostgreSQL full-text search [20], dense search, reciprocal rank fusion of the two lists, and finally cross-encoder re-ranking [15]. Retrieved text is fitted to a fixed budget so that a request respects the provider's per-minute token limit.

E. Verification and automatic correction

The verifier extracts every citation, quotation and statutory reference; normalises citations; resolves them against the case store; assigns a verdict (verified, partial, conflict or not found); checks quotations against the judgment text; asks a separate judge prompt whether the cited passage supports the sentence; inspects later treatment; and checks that each section exists and is in force on the relevant date. Automatic correction then replaces a wrong citation with the correct one when the named case exists, removes citations that cannot be grounded together with the sentence that relied on them, and removes case names written without a citation. The user sees only the repaired text and a one-line summary.

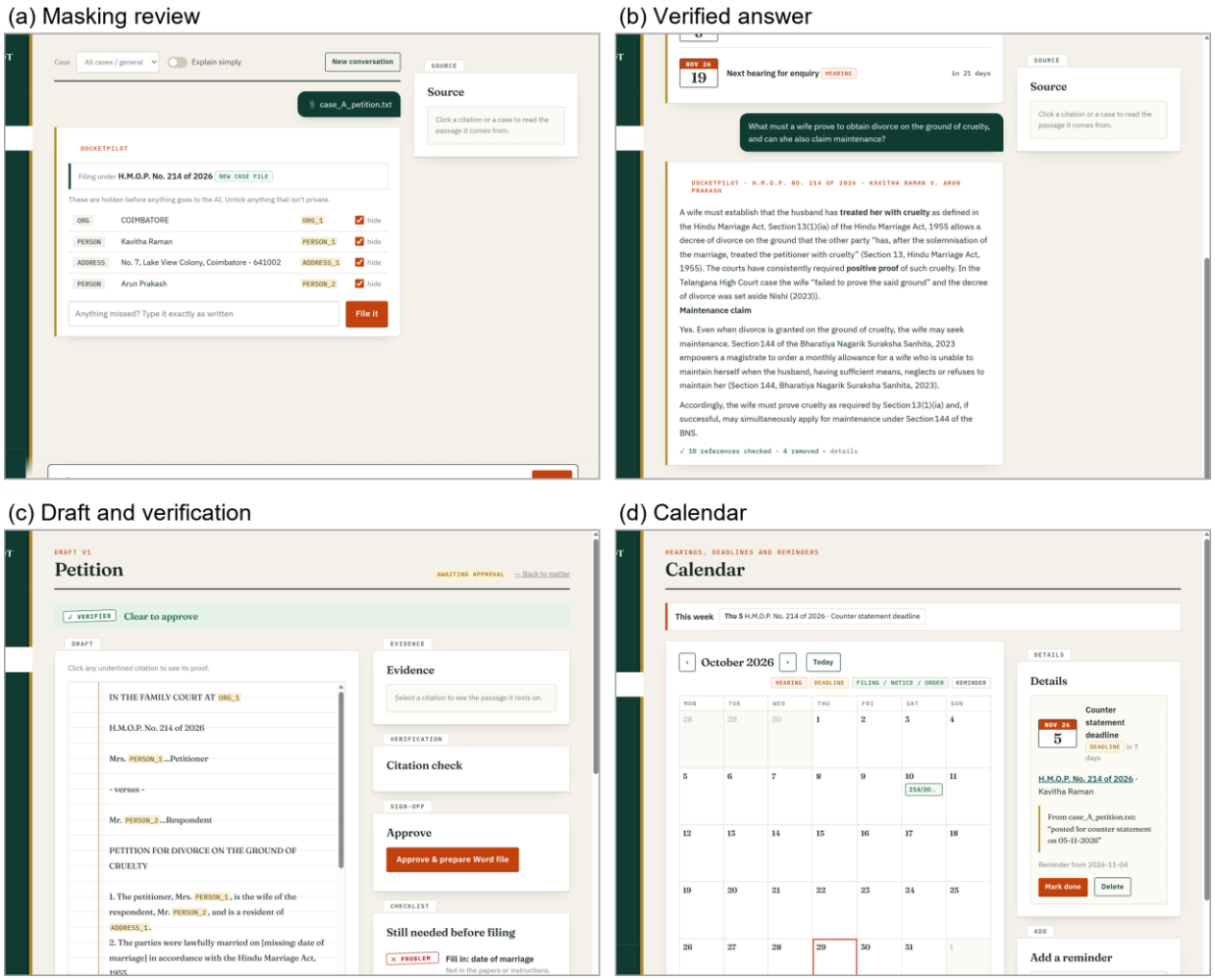


Fig. 2. Screens of the deployed system: (a) masked text reviewed before upload, (b) an answer whose citations have been verified, (c) a draft with its verification panel, (d) hearing dates extracted to the calendar.

IV. Experimental Setup

All experiments use one model, gpt-oss-120b on Groq, so that differences arise from the pipeline rather than the model. Costs use Groq list prices (US$0.15 and US$0.60 per million input and output tokens). We evaluate: (1) citation-error detection on 150 real Supreme Court cases, each corrupted in five ways (wrong page, wrong year, swapped party names, wrong court, invented case) plus six citations used for the wrong proposition, mixed with 156 genuine citations; (2)

retrieval on 93 queries in which a judgment's own headnote must retrieve its body; (3) masking on held-out synthetic Indian case papers; (4) eight research questions spanning criminal, family, contract, property, arbitration and consumer law, answered with and without the pipeline; and (5) eight real High Court judgments used as case papers. A citation counts as fabricated only if its format is one the store covers completely (Supreme Court neutral and SCR citations); other formats are reported as uncheckable rather than fabricated [6].

V. Results and Discussion

A. Effect of each layer

Fig. 3 adds one layer at a time to the same model. Answering from memory (A), 11.3% of checkable citations were fabricated and 4.8% named real cases that do not support the stated point. Supplying retrieved sources (B) lowered fabrication to 7.7% but raised misattribution to 15.4%: given real material, the model attaches it to claims it does not make. Requiring a source tag on every legal statement (C) removed fabrication entirely, yet the judge rejected 38.9% of tagged citations as not supporting the exact sentence. Only verification with automatic correction (D) eliminated both failure modes: of 20 citations written in the eight full answers, 11 were removed with the claims that relied on them and the 9 released were all verified. The layers are therefore complementary rather than cumulative: tagging addresses existence, verification addresses support. With eight questions the percentages are indicative; the pattern, not the exact values, is the finding.

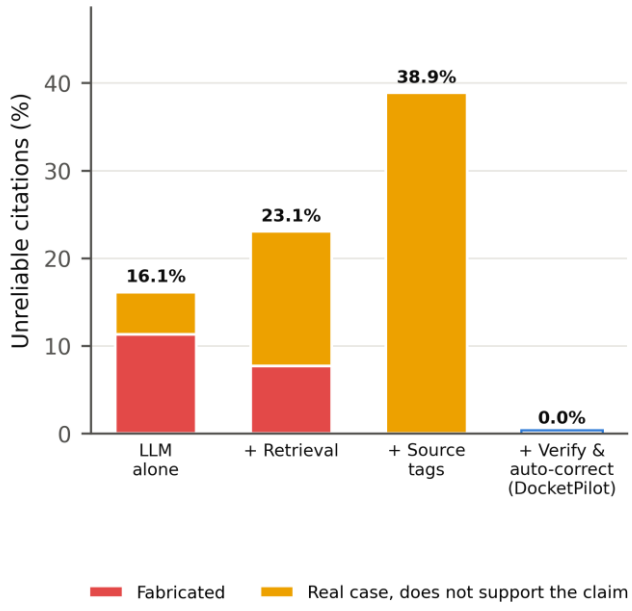


Fig. 3. Ablation on eight questions with the same model: unreliable citations (fabricated, or real but not supporting the claim) as each layer is added.

B. Comparison with published systems

Fig. 4 places the full pipeline beside published hallucination rates. The figures for other systems were obtained under different protocols and jurisdictions and are given for context only. Fig. 5 reports citation-error detection: on our benchmark the verifier reaches 99.6% recall and 99.3% precision. The same base model used as an agentic checker in [4] reaches 55.1% recall, so the gain comes from grounding verification in a complete case store rather than from the model itself. Swapped party names are the hardest corruption (98% caught), because the rest of the citation is correct; Fig. 6 gives the full confusion matrix.

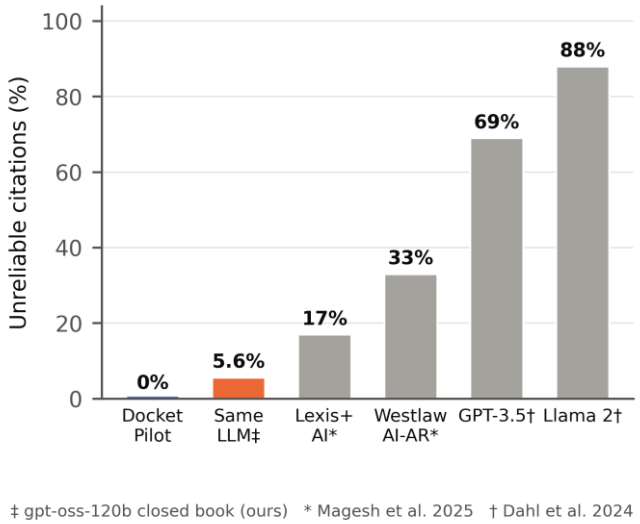


Fig. 4. Unreliable citations: DocketPilot and the same model answering from memory (measured), against published figures for commercial tools [2] and general models [3].

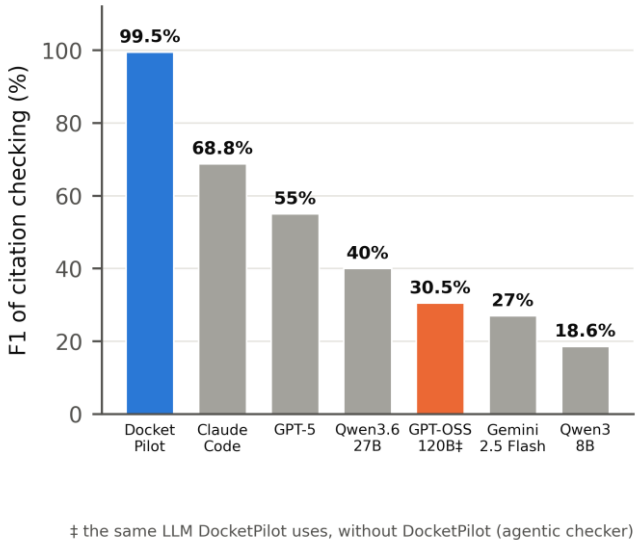


Fig. 5. F1 of citation-error detection. Other systems from [4] (agentic setting, US briefs); DocketPilot on the Indian benchmark.

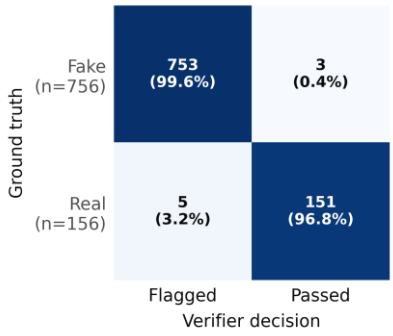


Fig. 6. Confusion matrix of the citation verifier on 756 fake and 156 genuine citations.

C. Cost and speed

Because only the best passages are sent, a verified research answer costs US$0.00285 against US$0.02215 when the same model reads the full judgments, a 87.2% reduction that includes every verification call (Fig. 7). Verification is not free: in the ablation run retrieval took 9.3 s on a laptop CPU, generation 2.7 s and verification 2.2 s per answer, and the verifier consumed 2,946 of the 6,337 input tokens per answer. Reading a case paper took 3.0 s versus 4.7 s for the unassisted model, a 35.9% saving that is smaller than the 60.8% reported for LegalMind [1]; DocketPilot trades raw speed for checking.

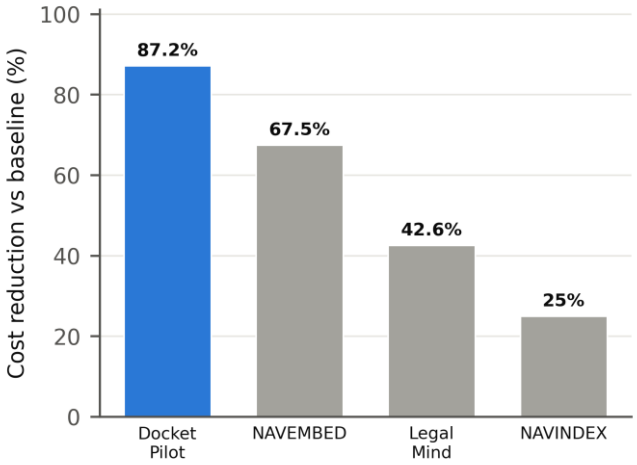


Fig. 7. Cost reduction against an AI baseline: DocketPilot (measured) and published systems [1], [8].

D. Retrieval and privacy

Fig. 8 confirms that neither lexical nor dense search suffices alone: Recall@5 rises from 88.2% for lexical search to 93.5% with fusion and 95.7% after re-ranking, at the cost of extra latency. The privacy gateway masked every structured identifier in the held-out set; overall recall was 96.8%, with person names the main source of misses, which the review screen is designed to catch.

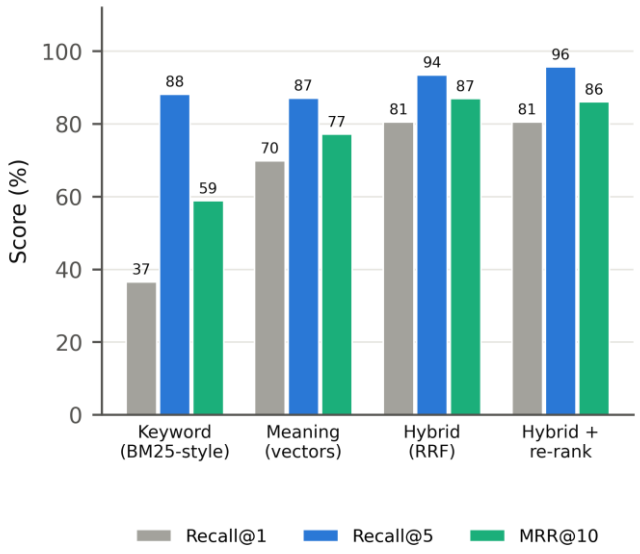


Fig. 8. Retrieval methods on 93 headnote-to-judgment queries.

TABLE II. COMPONENT RESULTS

Component	Metric	Result
Citation verifier	Recall / precision / F1	99.6 / 99.3 / 99.5%
Retrieval (hybrid + re-rank)	Recall@1 / Recall@5 / MRR@10	80.6 / 95.7% / 0.861
Privacy gateway (held-out)	Recall / leakage	96.8 / 3.2%
Statute date logic	Boundary checks passed	18 / 18
Released answers	Unreliable citations	0.0%

Fig. 9 sets these numbers beside the Indian systems that report comparable scores. The first passage returned is the correct judgment in 80.6% of queries, above the 74% retrieval precision of NyayaAI [26]; the ranking quality (MRR 0.861) exceeds the MAP of 0.36 reported by AQgR [27] (with one relevant judgment per query MAP equals MRR); and the verifier's F1 of 99.5% exceeds NyayaAI's headline F1 of 0.71. Test sets differ, so the comparison positions rather than ranks the systems.

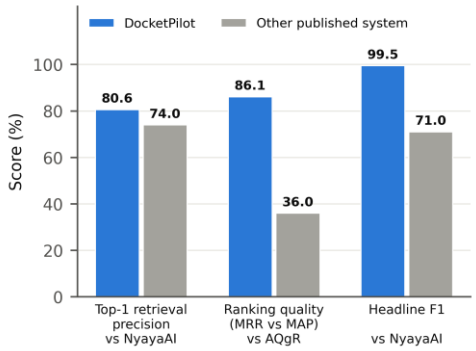


Fig. 9. DocketPilot against published Indian legal-AI systems on the scores they report [26], [27]; each system on its own test set.

E. Explainability

To make evidence selection inspectable we apply LIME [12] to the re-ranker: words of the selected passage are removed at random 400 times, each variant is scored, and a

weighted linear surrogate is fitted. Fig. 10 shows that, for a question on the presumption under Section 139 of the Negotiable Instruments Act, the decisive terms are the section number, “rebuttable” and the Act's name, while for a maintenance question they are “wife”, “husband”, “claim” and “maintenance”. The surrogate fits well (R^2 of 0.86 and 0.75), so a lawyer can see why a judgment was offered as support.

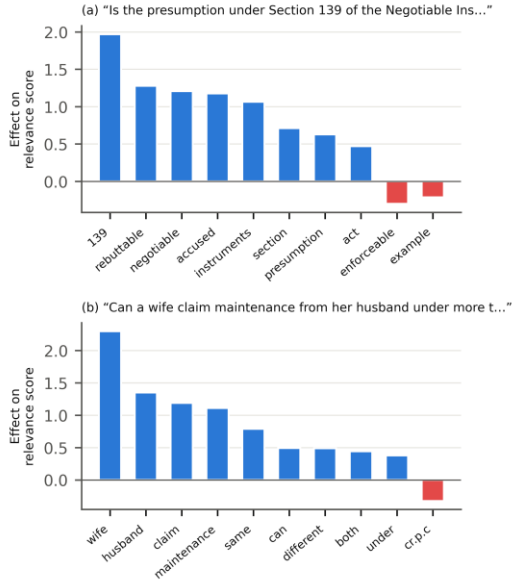


Fig. 10. LIME attributions of the evidence re-ranker for two queries; blue words raise and red words lower the relevance score.

VI. Limitations and Future Work

The evaluation is automatic: citation reliability is judged against a case store and an AI judge, and the question sets are small because of free-tier API quotas. Full-text coverage is limited to the Supreme Court from 2015 and a sample of five High Court benches; citations in formats the store does not fully cover cannot yet be checked. Scanned papers require OCR, which is not yet integrated. Planned work includes a study with practising advocates measuring time saved and output quality, wider High Court coverage, and hosting on a cloud server with encrypted storage.

VII. Conclusion

DocketPilot shows that a single open-weight model, wrapped in retrieval, source tagging and a verifier that repairs its own output, can deliver legal answers and drafts whose citations are checked before release, at a fraction of the cost of long-context prompting and without exposing client identities. Reducing unreliable citations from 16.1% to 0.0% required every layer of the pipeline, which suggests that verification should be treated as a first-class component of legal AI rather than an optional check.

References

[1] [Authors], “LegalMind: Agentic AI-driven process optimization and cost reduction in legal services using DeepSeek,” IEEE Access, vol. 13, 2025, doi: 10.1109/ACCESS.2025.3586781.

[2] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, “Hallucination-free? Assessing the reliability of leading AI legal research tools,” J. Empirical Legal Stud., vol. 22, pp. 216–242, 2025, doi: 10.1111/jels.12413.

[3] M. Dahl, V. Magesh, M. Suzgun, and D. E. Ho, “Large legal fictions: Profiling legal hallucinations in large language models,” J. Legal Anal., vol. 16, no. 1, pp. 64–93, 2024.

[4] P. Liu, D. Stammbach, and P. Henderson, “Who checks the citations? Benchmarking legal hallucination detection,” arXiv:2606.21155, 2026.

[5] S. Chen, H. Yin, and S. Zhou, “LegalCiteBench: Evaluating citation reliability in legal language models,” in Proc. ICML, 2026, arXiv:2605.10186.

[6] V. Ovcharov, “Citation grounding measures the oracle: Graph coverage determines reported LLM hallucination rates in law,” arXiv:2606.00898, 2026.

[7] J. Bose, “Falkor-IRAC: Graph-constrained generation for verified legal reasoning in Indian judicial AI,” arXiv:2605.14665, 2026.

[8] M. Hany, M. ElSheraey, M. Said, and P. Naoum, “Inject or navigate? Token-efficient retrieval for LLM analysis of transactional legal documents,” arXiv:2607.05764, 2026.

[9] L. Martin et al., “Better call GPT, comparing large language models against lawyers,” arXiv:2401.16212, 2024.

[10] Vals AI, “Vals legal AI report,” Feb. 2025. [Online]. Available: https://www.vals.ai

[11] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020, pp. 9459–9474.

[12] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’ Explaining the predictions of any classifier,” in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[13] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in Proc. ACM SIGIR, 2009, pp. 758–759.

[14] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packaged resources to advance general Chinese embedding,” arXiv:2309.07597, 2023.

[15] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

[16] N. Guha et al., “LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models,” in Proc. NeurIPS Datasets and Benchmarks, 2023.

[17] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[18] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” 2025.

[19] LangChain, “LangGraph: Building stateful, multi-actor applications with LLMs,” 2025. [Online]. Available: https://langchain-ai.github.io/langgraph/

[20] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333–389, 2009.

[21] Indian Supreme Court and High Court judgments, Registry of Open Data on AWS (CC-BY-4.0); India Code, Legislative Department, Government of India.

[22] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, “spaCy: Industrial-strength natural language processing in Python,” 2020.

[23] A. Joshi et al., “IL-TUR: Benchmark for Indian legal text understanding and reasoning,” in Proc. ACL, 2024.

[24] V. Malik et al., “ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation,” in Proc. ACL-IJCNLP, 2021, pp. 4046–4062.

[25] S. Paul, A. Mandal, P. Goyal, and S. Ghosh, “Pre-trained language models for the legal domain: A case study on Indian law,” in Proc. ICAIL, 2023, pp. 187–196.

[26] Deepanshu, D. Saxena, D. Rana, A. Varshney, and S. R. Laskar, “NyayaAI: An AI-powered legal assistant using multi-agent architecture and retrieval-augmented generation,” arXiv:2605.10155, 2026.

[27] V. Vishnuprabha, D. M. Viswanathan, R. Rajesh, and A. V. Pillai, “Augmented question-guided retrieval (AQgR) of Indian case law with LLM, RAG, and structured summaries,” arXiv:2508.04710, 2025.

[28] R. Goel, S. P. Kumar, A. Agrawal, D. Poddar, P. Narang, and D. Kumar, “Domain-partitioned hybrid RAG for legal reasoning: Toward modular and explainable legal AI for India,” arXiv:2602.23371, 2025.